

## ЛЕКЦИЯ 2

# Дискретно-событийные системы: языки, автоматы, композиция

*Как из моделей отдельных единиц оборудования вычислить модель ячейки*

две части по 45

ение робототехническими комплексами»

# План занятия

- 1 Смена точки зрения: автомат порождает язык
- 2 События, трассы, языки, префиксное замыкание
- 3 Генератор: два языка и что между ними
- 4 Блокировка: тупик и живой замок
- 5 Критерий неблокируемости и его доказательство

- 6 Проекция и её свойства
- 7 Параллельная композиция: главная операция курса
- 8 Теорема о языке композиции
- 9 Сборка модели ячейки и первый найденный тупик
- 10 Наблюдаемость и диагностируемость отказов

*Синие — первая часть · янтарные — вторая часть*

ЧАСТЬ 1

# Автомат как генератор языка

*Переопределяем знакомый объект так, чтобы им можно было моделировать комплекс.*

### Базовый курс

- Автомат преобразует слово
- Вопрос: какой выход при данном входе?
- Главная операция — минимизация
- Модель — устройство
- Один автомат

### Теория ДЭС

- Автомат порождает язык
- Вопрос: какие трассы возможны?
- Главные операции — композиция, синтез
- Модель — процесс, ограничиваемый супервизором
- Много взаимодействующих автоматов

“

Модель ячейки не рисуют — её вычисляют композицией моделей компонентов.

*Пятьдесят тысяч состояний из восьми автоматов по три-шесть состояний*

## 2.2 Событие: что им является и что нет

- Событие мгновенно, атомарно, наблюдается как факт
- ``start_M1`, `pick_done_M1`, `part_arrived`, `machine_done`, `fail_M2``
- «Манипулятор перемещается» — **состояние**, а не событие
- Событиями являются «начал перемещаться» и «закончил»
- Каждой продолжительной операции соответствуют минимум два события

### Ключевая идея

Смешение состояний и событий — типовая ошибка моделирования. Она приводит к моделям, в которых невозможно выразить длительность операции и её прерывание, то есть невозможно описать таймаут и отмену.

$$\bar{L} = \{ u \in \Sigma^* : \exists v \in \Sigma^*, uv \in L \}$$

- $\Sigma$  — алфавит событий,  $\Sigma^*$  — все конечные трассы
- Язык  $L \subseteq \Sigma^*$  — множество возможных последовательностей
- $\bar{L}$  — все префиксы всех трасс  $L$
- Поведение реальной системы всегда префиксно-замкнуто

Незамкнутые языки описывают задания, а не поведение

## 2.4 Генератор: определение

- $G = (X, \Sigma, f, \Gamma, x_0, X_m)$
- $f : X \times \Sigma \rightarrow X$  — **частичная** функция переходов
- $\Gamma(x)$  — множество активных в  $x$  событий
- $X_m$  — маркированные состояния: завершение задачи
- $L(G)$  — всё возможное поведение;  $L_m(G)$  — завершённые задачи

### Ключевая идея

Частичность функции переходов — не дефект модели. Она означает, что событие физически невозможно в данном состоянии. Добавление «ловушки», как при минимизации акцепторов, здесь вредно: оно скрывает информацию о физической невозможности.

## 2.5

## Блокировка: два вида

- Автомат неблокирующий, если  $L_m(G) = L(G)$
- **Тупик**: достигнуто состояние без активных событий
- **Живой замок**: цикл, из которого не достичь маркированного состояния
- Тупик виден оператору немедленно — всё стоит
- Живой замок выглядит как «работает, но продукция не выходит»

### Кейс

Живой замок опаснее тупика: линия гудит, роботы двигаются, индикаторы зелёные, но заказы не закрываются. Без формальной модели такой дефект диагностируется неделями.

**G неблокирующий  $\Leftrightarrow$  все достижимые состояния коаккуратны**

- Состояние коаккуратно, если из него достижимо маркированное
- Проверка: поиск в ширину от  $x_0$  и обратный поиск от  $X_m$
- Сложность  $O(|X| + |\delta|)$  — линейная
- Свойство, не проверяемое тестированием, проверяется линейным алгоритмом

При условии, что модель построена — а строится она композицией



## Проверим понимание

Станок:  $I \rightarrow s\_start \rightarrow W \rightarrow s\_fail \rightarrow F$ , и перехода  $s\_repair$  из  $F$  нет. Состояние  $I$  маркировано. Что можно сказать о модели?

A

Модель корректна: отказ так отказ

B

Модель блокирующая:  $F$  не коаккуратно

C

Модель некорректна синтаксически

D

Нужно добавить в  $F$  петлю



## Проверим понимание

Станок:  $I \rightarrow s\_start \rightarrow W \rightarrow s\_fail \rightarrow F$ , и перехода  $s\_repair$  из  $F$  нет. Состояние  $I$  маркировано. Что можно сказать о модели?

A

Модель корректна: отказ так отказ

B

Модель блокирующая:  $F$  не коаккуратно

C

Модель некорректна синтаксически

D

Нужно добавить в  $F$  петлю

**Ответ.** Модель блокирующая: из  $F$  не достичь маркированного состояния. Формальная модель немедленно выявляет то, что в неформальном описании выглядело допустимым.

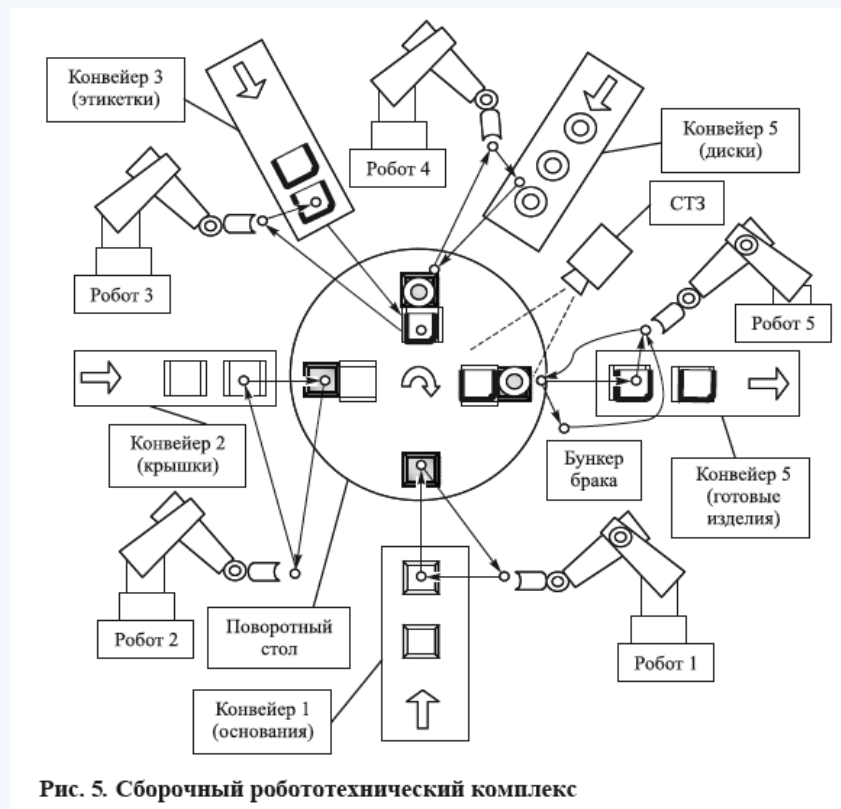
- $P : \Sigma \rightarrow \Sigma_s$ , стирает события вне подалфавита
- $P(\sigma) = \sigma$  при  $\sigma \in \Sigma_s$ , иначе  $P(\sigma) = \varepsilon$
- Обратная проекция  $P^{-1}$  действует на языки
- $P^{-1}(P(L)) \supseteq L$ , обычно строго — это и есть потеря информации
- Две роли: частичная наблюдаемость и локальный взгляд компонента

**$G_1 \parallel G_2$  : общие события синхронны, частные независимы**

- $\sigma \in \Sigma_1 \cap \Sigma_2$  — происходит, только если разрешено в обоих
- $\sigma \in \Sigma_1 \setminus \Sigma_2$  — происходит, второй автомат «не замечает»
- Общие события — единственный канал взаимодействия компонентов
- Операция коммутативна и ассоциативна с точностью до изоморфизма

Проектирование модели ячейки = правильный выбор общих событий

## 2.8 Реальная сборочная ячейка



Пять манипуляторов, пять конвейеров, поворотный стол, СТЗ и бункер брака. Тринадцать компонентов - модель такой ячейки руками не рисуют.

$$L(G_1 \parallel G_2) = P_1^{-1}(L(G_1)) \cap P_2^{-1}(L(G_2))$$

- Трасса возможна в композиции  $\Leftrightarrow$  её проекции возможны в компонентах
- Доказательство — индукция по длине трассы, три случая события
- Даёт строгое основание для модульного анализа
- При  $\Sigma_1 = \Sigma_2$  композиция вырождается в произведение:  $L_1 \cap L_2$

## 2.10 Модели компонентов ячейки

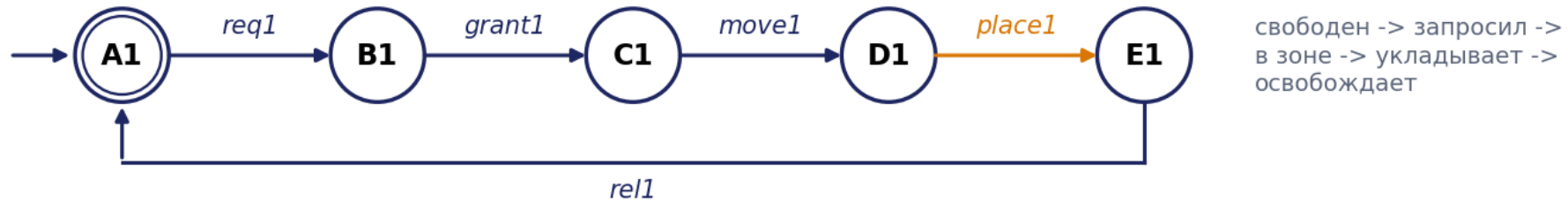
| Компонент         | Состояния                 | Алфавит                              |
|-------------------|---------------------------|--------------------------------------|
| Манипулятор $M_1$ | $A_1 B_1 C_1 D_1 E_1$ (5) | $req_1 grant_1 move_1 place_1 rel_1$ |
| Станок $S$        | $I L W R$ (4)             | $place_1 s\_start s\_done s\_unload$ |
| Зона $Z$ (ресурс) | $Free Busy_1 Busy_2$ (3)  | $grant_1 rel_1 grant_2 rel_2$        |
| Манипулятор $M_2$ | $A_2 B_2 C_2 D_2 E_2$ (5) | $req_2 grant_2 unload_2 rel_2$       |

Событие  $place_1$  общее для  $M_1$  и  $S$  — это и есть точка синхронизации

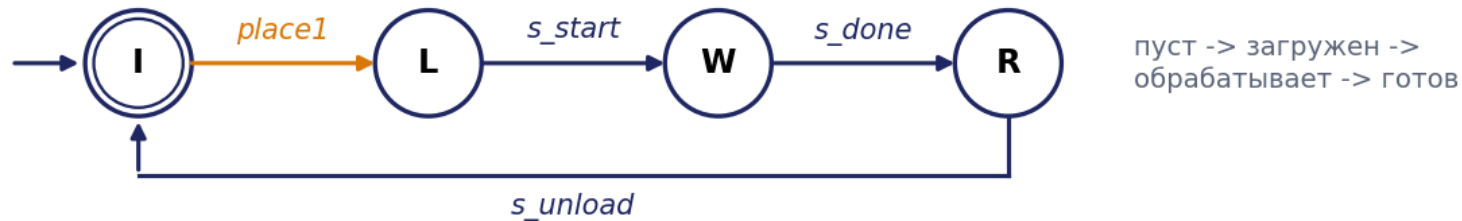
### Модели компонентов ячейки: манипулятор M1 и станок S

*Событие place1 принадлежит обоим алфавитам - это единственная точка их синхронизации*

#### Манипулятор M1



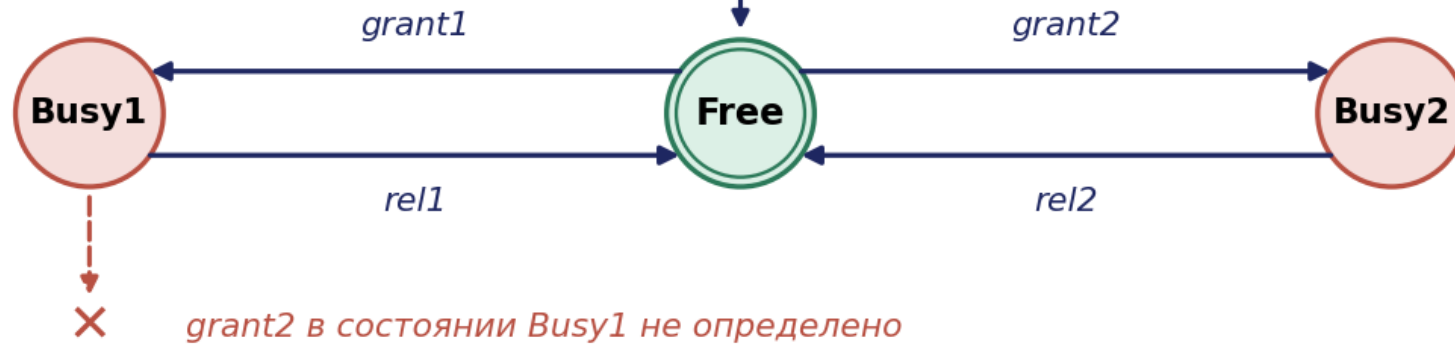
#### Станок S



Событие place1 принадлежит обоим алфавитам - это единственная точка синхронизации манипулятора и станка.

## 2.10 Зона Z: ресурс - тоже автомат

Зона Z: ресурс моделируется отдельным автоматом



Взаимное исключение не программируется и не проверяется в коде.  
Оно следует из структуры автомата ресурса, а значит сохраняется в любой композиции.

Взаимное исключение следует из структуры автомата, а не из проверок в коде, и потому сохраняется в любой композиции.

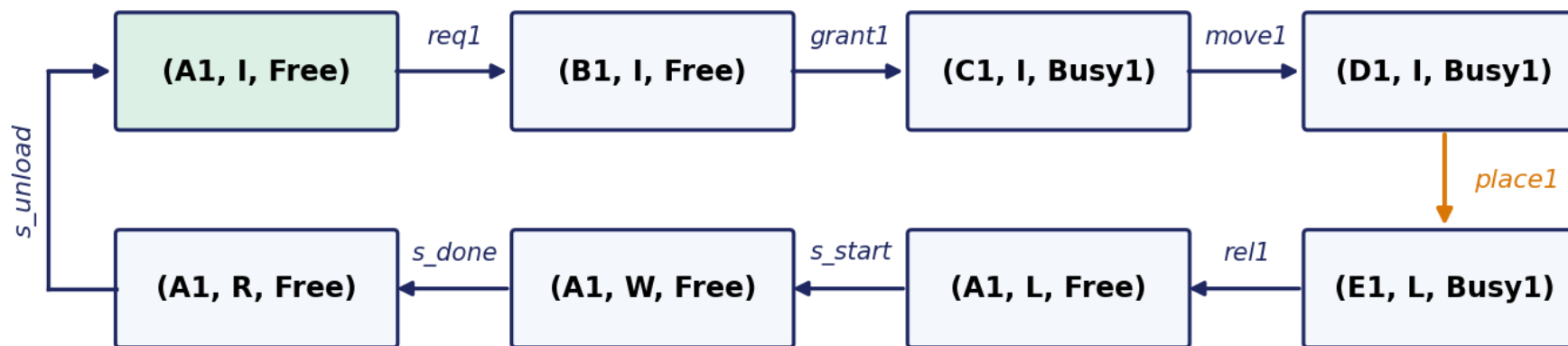
## 2.11 Прогон композиции: один цикл



Верхняя оценка  $5 \cdot 4 \cdot 3 = 60$ , фактически достижимо существенно меньше

## 2.11 Один цикл в композиции

### Один производственный цикл в композиции M1 || S || Z



Верхняя оценка:  $5 \times 4 \times 3 = 60$  состояний.  
Достижимо существенно меньше

Эта цепочка не нарисована, а вычислена  
операцией композиции

Цепочка не нарисована, а вычислена. Верхняя оценка 60 состояний, достижимо существенно меньше.

## 2.12 Первый найденный тупик

- Состояние ( $C_1, B_2, L, Busy_1$ ) —  $M_1$  в зоне, хочет уложить
- Станок уже загружен ( $L$ ) — событие  $place_1$  заблокировано
- $M_2$  ждёт зону, зона занята  $M_1$
- $s\_start$  требует свободной зоны — тоже невозможно
- Активных событий нет: **тупик**

### Факт

Алгоритм нашёл это за один прогон. В реальной ячейке такой сценарий проявляется раз в несколько тысяч циклов и списывается на «сбой», для которого нет воспроизводимого сценария.

## 2.12 Тупик, найденный за один прогон



В реальной ячейке такой сценарий проявляется раз в несколько тысяч циклов.



## Как исправить?

Какое исправление тупика вы считаете правильным для ячейки из десяти компонентов?

A

Добавить проверку в предусловие `place1`

B

Ввести буфер между роботом и станком

C

Синтезировать супервизор автоматически

D

Поставить таймаут и перезапускать ячейку



## Как исправить?

Какое исправление тупика вы считаете правильным для ячейки из десяти компонентов?

A

Добавить проверку в предусловие `place1`

B

Ввести буфер между роботом и станком

C

Синтезировать супервизор автоматически

D

Поставить таймаут и перезапускать ячейку

**Ответ.** Третий. Первые два — ручные правки, каждая из которых может породить новый тупик; четвёртый маскирует проблему. Синтез гарантирует отсутствие тупиков по построению.

П Е Р Е Р Ы В

## Конец первой части

*Мы научились собирать модель ячейки и проверять её на блокировки. После перерыва — что делать, когда не все события наблюдаемы.*

## ЧАСТЬ 2

# Наблюдаемость и диагностика

*Что можно узнать о состоянии системы, видя не все события, и достаточно ли установленных датчиков.*

## 2.13 Частичная наблюдаемость

- $\Sigma = \Sigma_o \cup \Sigma_{uo}$  — наблюдаемые и ненаблюдаемые события
- Ненаблюдаемы: внутренние переходы, срыв захвата без датчика, дрейф
- Управляющая система видит проекцию  $P(s)$ , а не саму трассу  $s$
- Одной наблюдаемой трассе соответствует **множество** возможных состояний
- Оценка состояния  $B(t) = \{ f(x_o, s) : P(s) = t \}$

$$|Q(\text{Obs})| \leq 2^{|X|}, \text{ и оценка достижима}$$

- Состояния наблюдателя — подмножества  $X$
- Построение аналогично детерминизации с  $\epsilon$ -переходами
- Роль  $\epsilon$  играют ненаблюдаемые события
- Экспоненциальный рост — принципиальное препятствие

Состав наблюдаемых событий — проектное решение, а не данность

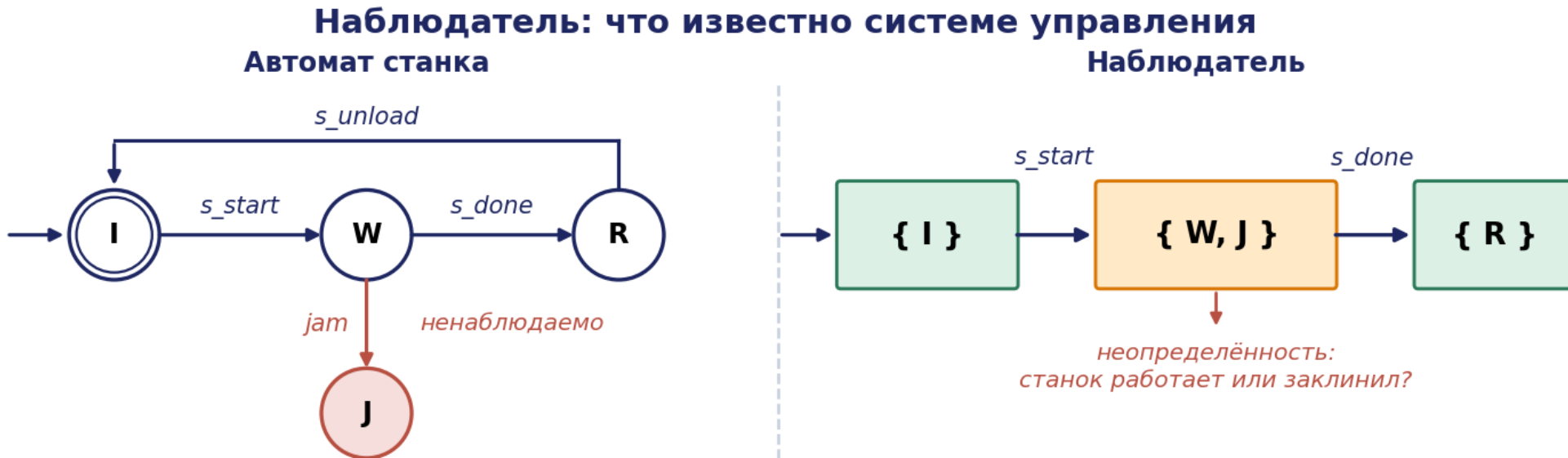
## 2.15 Пример: станок без датчика заклинивания

- Состояния: I, W, R, J (заклинило); событие `jam` ненаблюдаемо
- После наблюдения `s\_start` оценка состояния = {W, J}
- Система не знает, работает станок или заклинил
- Никакая логика не может различить эти случаи
- Значит, не может и корректно среагировать

### Пример

Два решения: добавить датчик, сделав `jam` наблюдаемым, либо ввести наблюдаемое событие `timeout`. Таймаут — типовой приём: он превращает ненаблюдаемое событие в наблюдаемое с задержкой.

## 2.15 Наблюдатель при ненаблюдаемом отказе



Никакая логика управления не различит W и J, а значит не среагирует на заклинивание.  
Решения два: датчик, делающий jam наблюдаемым, либо таймаут - наблюдаемое событие с задержкой

Оценка состояния {W, J}: никакая логика не различит работу и заклинивание.

## 2.16 Диагностируемость отказа

- Отказ моделируется ненаблюдаемым событием  $\sigma_f$
- Язык диагностируем, если отказ достоверно устанавливается за  $n$  шагов
- Диагностёр — наблюдатель с метками  $Y$  (был отказ) и  $N$  (не было)
- Состояние неопределённо, если содержит и  $Y$ , и  $N$
- Критерий: отсутствие **неопределённого цикла**

### Ключевая идея

Если язык не диагностируем, никакое усовершенствование алгоритма диагностики не поможет — информации физически нет. Требуется изменение состава сенсоров.

## ? Подумаем

В ячейке отказ «деталь выпала из захвата при транспортировке» ненаблюдаем. Какое минимальное изменение делает его диагностируемым?

A

Улучшить алгоритм оценки массы по току привода

B

Добавить датчик наличия детали в захвате

C

Увеличить частоту опроса имеющихся датчиков

D

Добавить контроль веса на приёмной позиции

## ? Подумаем

В ячейке отказ «деталь выпала из захвата при транспортировке» ненаблюдаем. Какое минимальное изменение делает его диагностируемым?

**A** Улучшить алгоритм оценки массы по току привода

**B** Добавить датчик наличия детали в захвате

**C** Увеличить частоту опроса имеющихся датчиков

**D** Добавить контроль веса на приёмной позиции

**Ответ.** Второй или четвёртый — оба вводят новое наблюдаемое событие. Третий не помогает: чаще опрашивать то, что не несёт информации, бесполезно.

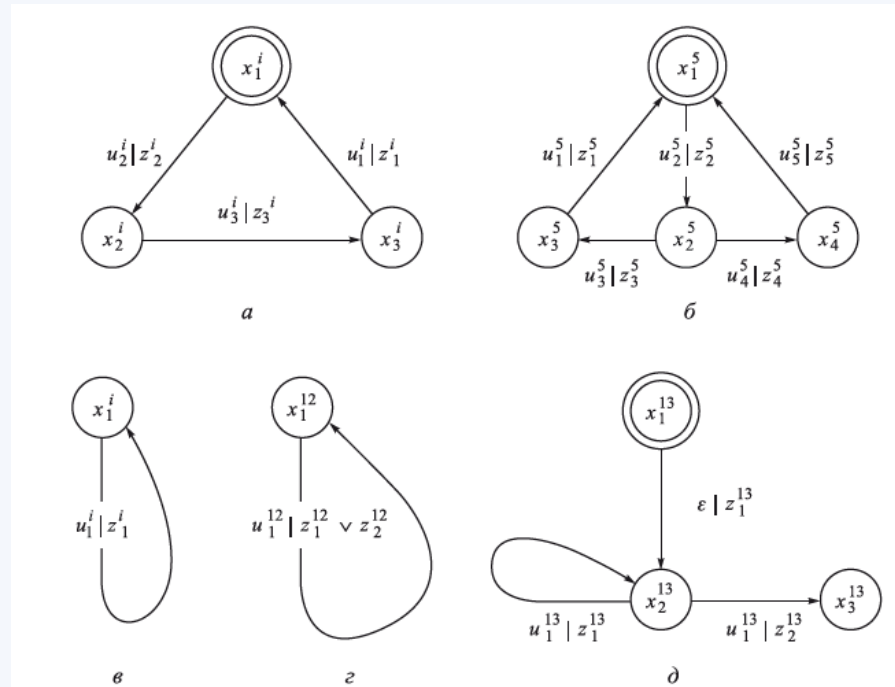
## 2.17

## Восемь правил построения ДЭС-модели

| Правило | Содержание                                         |
|---------|----------------------------------------------------|
| 1       | Один компонент — один автомат; ресурсы тоже        |
| 2       | События — переходы, а не состояния                 |
| 3       | Взаимодействие только через общие события          |
| 4       | Ресурс моделируется явно: Free/Busy                |
| 5       | Маркированные состояния = завершение задачи        |
| 6       | Отказы включаются в модель с самого начала         |
| 7       | Наблюдаемость фиксируется явно для каждого события |
| 8       | Проверять неблокируемость после каждой композиции  |

Правило 8 — самое дешёвое: проверка линейна, а ошибка обнаруживается рано

## 2.17 Те же правила в инженерной практике

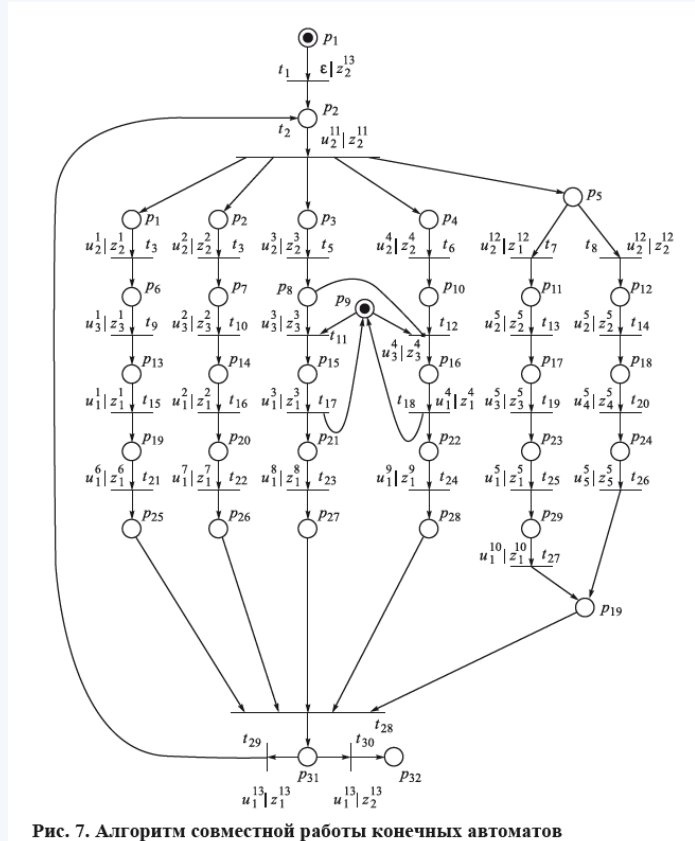


**Рис. 6. Модели объектов:**

$a - A_i$  – модель робота  $i$ ,  $i=1, \dots, 4$ ;  $б - A_5$  – модель робота 5;  $в - A_i$ ,  $i=6, \dots, 10$ , – модель конвейера с номером  $(i-5)$ ;  $i=11$  – модель поворотного стола;  $г - A_{12}$  – модель СТЗ;  $д - A_{13}$  – модель системы управления верхнего уровня

Автомат на компонент, события - переходы, СТЗ и верхний уровень моделируются наравне с оборудованием.

## 2.17 Совместная работа тринадцати автоматов



32 позиции и 30 переходов для одной сборочной ячейки. Именно этот объём и заставляет переходить от рисования к вычислению.

## 2.18 Реализация: композиция в тридцать строк

- Обход от начального состояния, три случая на событие
- Общее событие — оба перехода должны существовать
- Частное событие — переход одного, второй не меняется
- Маркированные состояния — пары маркированных
- Проверка неблокируемости — обратный поиск от  $X_m$

### Ключевая идея

Функция проверки должна возвращать не только флаг, но и множество блокирующих состояний — именно оно является отладочным артефактом, из которого восстанавливается сценарий тупика.

## 2.19 Инструменты для работы с ДЭС

| Инструмент                | Назначение                                      |
|---------------------------|-------------------------------------------------|
| Supremica                 | синтез и верификация, символьные алгоритмы, GUI |
| DESUMA / UMDES            | операции над ДЭС, наблюдатели, диагностёры      |
| TCT                       | классический супервизорный синтез               |
| libFAUDES                 | открытая библиотека C++                         |
| Своя реализация на Python | понимание, недостижимое при нажатии кнопки      |

На семинаре 2 — Supremica плюс собственная реализация композиции

## Три способа бороться с комбинаторным взрывом

- **Модульность:**  $n$  небольших супервизоров вместо одного гигантского
- **Абстракция:** проекция с сохранением свойства наблюдателя
- **Символьное представление:** BDD, до  $10^{20}$  состояний и более
- Модульность требует проверки неконфликтности — глава 3
- Символьный анализ подробно — глава 7

## 2.21 Что даёт формальная модель: цифры

$$O(|X| + |\delta|)$$

сложность проверки  
неблокируемости

34 650

чередований у 3 компонентов  
по 4 шага

1

прогонов алгоритма для поиска  
тупика

$$2^{|X|}$$

верхняя оценка размера  
наблюдателя

- Модель строится час, проверка выполняется за секунды
- Тот же дефект в эксплуатации ищется неделями

“

Состав датчиков определяет, какие задачи управления вообще разрешимы.

*Поэтому сенсорика проектируется вместе с логикой, а не до неё*

# Выводы

- 1 ДЭС-модель описывает порождаемый язык, а не преобразование входа в выход
- 2  $L(G)$  — всё возможное;  $L_m(G)$  — завершённые задачи; их соотношение даёт неблокируемость
- 3 Неблокируемость эквивалентна коаккуратности и проверяется за линейное время
- 4 Модель комплекса вычисляется композицией; общие события — единственный канал взаимодействия
- 5  $L(G_1 || G_2) = P_1^{-1}(L_1) \cap P_2^{-1}(L_2)$  — основа модульного анализа
- 6 Размер наблюдателя экспоненциален; состав датчиков — проектное решение
- 7 Диагностируемость определяет достаточность сенсорной оснастки до изготовления комплекса

# Вопросы для самопроверки

1. Почему функция переходов частичная и почему ловушка нежелательна?
2. Приведите систему, где  $L_m(G)$  строго меньше  $L(G)$ , и объясните физический смысл
3. Чем живой замок опаснее тупика для эксплуатации?
4. Сформулируйте и докажите критерий неблокируемости
5. Чем параллельная композиция отличается от произведения? Когда совпадают?
6. Докажите  $P(L_1 \cap L_2) \subseteq P(L_1) \cap P(L_2)$  и приведите пример строгого включения
7. Как моделируется ресурс и почему взаимное исключение возникает «само собой»?
8. Почему размер наблюдателя может быть экспоненциальным?
9. Что означает неопределённый цикл в диагностёре с инженерной точки зрения?
10. Почему таймаут можно считать способом сделать событие наблюдаемым?